Warhammer Kill Team Simulator

Viktor Vyhnalek
ČVUT–FIT
vyhnavik@fit.cvut.cz

19. května 2026

1 Úvod

Warhammer Kill Team[3] je komplexní šarvátková stolní hra, kde mezi sebou bojují týmy elitních operativů. V boji využívají své unikátní zbraně, charakterová pravidla a speciální vlastnosti. Pravidla obsahují spoustu unikátních modifikací a různých pravidel ovlivňující výsledky hodu kostkou. Kill team je složitá a taktická hra, proto jsem chtěl navrhnout program, který může hráčům pomoct orientovat se lépe ve hře. Implementoval jsem simulaci pomocí rozhodovacích stromů, kde se utráci *Action Pointy*, simuluji střelbu i boje na blízko v různých situacích. Hráč pak dostane v jednoduchém GUI šest tahů, které jsou z jeho pozice ideální.

2 Vstupní data

Projekt je navrhnut pro Kill Team Starter set[2], simulace používá oficiální pravidla a data sheety[1]. Tato data jsem přepsal do JSONu a následně jsem jimi vyplnil SQLite[6] databázi, se kterou dále pracuji. Také jsem musel implementovat mnoho speciálních pravidel, které ovlivňují hody kostkou. Nicméně hody kostkou nesimuluji, ale počítám pravděpodobnosti hodnot kostek (např. úspěšná trefa na 3+). Simulátor průběžně s házením započítá modifikátory a použije výsledné hodnoty pro rozhodování podle taktických cílů.

3 Jak pracuje algoritmus?

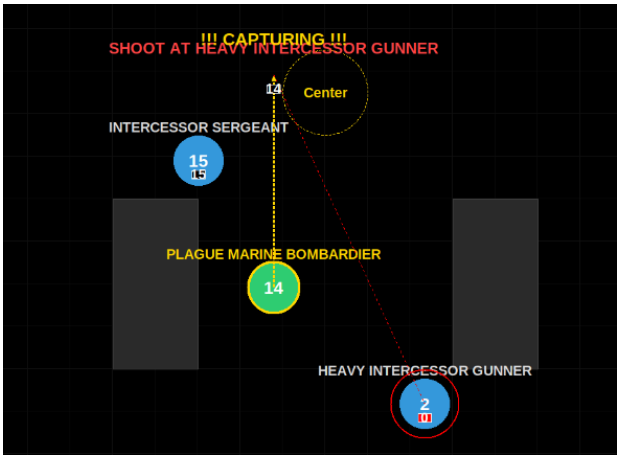
Simulátor pracuje na základě stavového hledání v prostoru, které je vylepšené o prvky AI. Cyklus je rozdělen do šesti částí:

1. **Zobrist Hashing[8] & Transposition Tables:** Před samotným hledáním se inicializuje hashovací tabulka. Každý unikátní stav hry (kombinace pozic, zbývajících životů a stavu aktivací) dostane svůj 64bitový klíč pomocí XOR operací. Tento přístup umožňuje ukládat vyhodnocené uzly do tzv. Transposition Table[10], což eliminuje duplicitní výpočty pro stejné pozice, do kterých se modely dostaly jinou sekvencí pohybů. Díky optimalizaci hashování, které ignoruje historii tahů, je vyhledávání velmi rychlé.
2. **Goal-Oriented Space Discretization:** Abych zabránil explozi stavového prostoru, mapa není brána jako spojitá mřížka. Algoritmus dynamicky generuje konečný počet strategických bodů v závislosti na zvoleném cíli (např. body kolem objektivů pro `CAPTURE`). K prostorových operacím využívám knihovnu Shapely[4].
3. **Heuristic Beam Search[9]:** Proces aktivace operativce tvoří stavový automat. Ve fázi pohybu rychlá geometrická heuristika (hodnotící kryt a vzdálenosti) vybere jen fixní počet nejslibnějších stavů. Ostatní uzly jsou zahozeny, což udržuje výpočetní náročnost v rozumných mezích.
4. **Action Expansion & P25 Trimmed Mean Evaluator:** U uzlů, které postoupí do akční fáze, engine simuluje pravděpodobnostní výsledky soubojů (`ShootPhase`, `FightPhase`). Místo prosté střední hodnoty počítám s odhadem na úrovni 25. percentilu. To zajišťuje, že doporučené tahy jsou spolehlivé a nejsou založeny na statistických extrémech. V této fázi probíhá také přísná kontrola viditelnosti (Line of Sight).
5. **Time-Bounded Iterative Deepening:** Hledání nepracuje s fixní hloubkou. Místo toho algoritmus postupně otevírá další patra rozhodovacího stromu, dokud nenarazí na časový limit (20 s). Engine dokáže simulovat komplexní sekvence akcí (např. pohyb, střelba a následná interakce s objektem) do hloubky několika tahů.
6. **Multi-Criteria Decision Analysis:** Finální vyhodnocení listových uzlů probíhá pomocí lineární užitkové funkce. Vstupy jsou normalizované proměnné (udělené poškození, vlastní životy, strategická pozice). Váhy parametrů

jsou pevně spjaty s úkolem – díky funkčnímu prohledávání do hloubky engine přirozeně upřednostňuje dlouhodě výhodné cíle (zabrání bodu) před okamžitým ziskem (střelba).

4 Výsledky

Simulátor je funkční a podařilo se mi vyřešit problémy, jako je střelba skrz zdi, přesné počítání vzdáleností nebo kolize modelů, které by jinak stály na sobě.



Obrázek 1: Ukázka simulace s motivem KILL, kde algoritmus úspěšně vyhodnocuje situaci a volí nejoptimálnější cíl pro útok.

Původně jsem musel používat velmi vysoké tresty pro akce, které nevedly k hlavnímu cíli, aby se simulace chovala rozumně. Po zprovoznění prohledávání do hloubky a opravě Line of Sight se však ukázalo, že algoritmus dokáže strategicky plánovat i bez těchto umělých omezení. Simulace nyní vidí několik kroků dopředu a dokáže se rozhodnout, kdy je výhodnější se pohnout do krytu a kdy riskovat útok. Matematicky algoritmus pracuje přesně a doporučené tahy v GUI odpovídají většinou reálné taktice v Kill Teamu.

5 Závěr

Na začátku jsem si asi ukousl příliš velké sousto. Téma Kill Teamu je skvělé, ale pravidla jsou extrémně komplexní. Musel jsem tedy omezit rozsah pouze na Starter Set. S implementací některých složitějších částí mi pomohla i AI (Gemini), protože stihnout vše v limitu 90 hodin by jinak absolutně nebylo realistické. Nakonec se mi ale podařilo dosáhnout cíle a s výsledkem jsem velmi spokojený. Vždy je co zlepšovat, například přidat pravidla pro plnou verzi hry nebo více testovat extrémní herní situace s profi hráči, abychom prakticky ověřili schopnosti mého engine. Celkově mám radost z toho, co jsem postavil a věřím, že by to mohla být užitečná aplikace.

Reference

[1] Games Workshop Ltd. Kill Team Starter Handbook. paperback, 2024. [cit. 2026–04–10].

[2] Games Workshop Ltd. Warhammer Kill Team Starter Set. online, 2024. [cit. 2026–05–16] https://start-warhammer.com/kill-team/.

[3] Games Workshop Ltd. Kill Team Official Website. online, 2026. [cit. 2026–05–16] https://www.warhammer-community.com/en-gb/topics/kill-team/.

[4] Sean Gillies et al. Shapely: Manipulation and analysis of geometric objects. online, 2024. [cit. 2026–05–16] https://shapely.readthedocs.io/.

[5] Google. Gemini 3.0: Large Language Model. AI Tool, 2026. Used for code implementation assistance and technical documentation refinement based on user algorithmic design.

[6] SQLite Development Team. SQLite Documentation. online, 2024. [cit. 2026–05–16] https://www.sqlite.org/docs.html.

[7] Sebastian Szczepański. Introduction to Ray-casting in 2D Game Engines. online, 2025. [cit. 2026–05–16] https://sszczep.dev/blog/ray-casting-in-2d-game-engines.

[8] Lars Wächter. Zobrist Hashing. online, 2020. [cit. 2026–05–16] https://dev.to/larswaechter/zobrist-hashing-72n.

[9] Wikipedia contributors. Beam Search Algorithm. online, 2026. [cit. 2026–05–16] https://en.wikipedia.org/wiki/Beam_search.

[10] Wikipedia contributors. Transposition Table. online, 2026. [cit. 2026–05–16] https://en.wikipedia.org/wiki/Transposition_table.